



PHP Programming

Fundamentals

Day 02

Prepared by: Mona Ali

Teaching Assistant — Information Technology Institute (ITI)

Agenda

- 01 Files in PHP
- 02 Open, Read, Write & Close Files
- 03 File Modes & Useful Functions
- 04 Arrays — Indexed & Associative
- 05 Array Sorting, Navigation & Functions
- 06 Lab 02 — Registration Form



Files in PHP

Reading, writing, and managing files



Storing Data

You can store data in two basic ways:

- 1. In flat files (simple text files)
- 2. In a database (MySQL, PostgreSQL, etc.)

Flat Files

Simple text files, CSV, logs.
Easy but limited for large data.

Database

Structured, searchable, secure.
Best for complex applications.

Opening the File

To open a file in PHP, use the `fopen()` function.

You must specify how you intend to use it — this is the file mode.

```
fopen(string $filename, string $mode) → Resource
```

PHP



`fopen()` returns a resource handle on success, or false on failure And gives a warning.

```
$fh = fopen("welcome.txt", "r");  
var_dump($fh);  
echo gettype($fh);
```

```
@$file2 = fopen("abc.txt", "r");  
if ($file2) {  
    echo "File exists";  
} else {  
    echo "File not found";  
}
```

PHP



File Modes

Mode	Name	Description
r	Read	Open for reading from start of file
r+	Read	Open for reading and writing from start
w	Write	Open for writing from start. Overwrite existing content
w+	Write	Open for writing and reading from start. Overwrite
x	Cautious	Writing only. Fail if file already exists
x+	Cautious	Writing and reading. Fail if file exists
a	Append	Open for writing at end of existing contents
a+	Append	Open for writing and reading from end



Use 'b' mode with others for binary files (recommended for portability). 't' is for text mode on Windows only.

Processing Files (Reading) - fread()

Reading data from a file requires three steps:

1. Open the file (handle missing files gracefully)

2. Read the data from the file

3. Close the file

```
$fh = fopen("welcome.txt", "r");  
$data = fread($fh, filesize("welcome.txt"));  
var_dump($data);  
fclose($fh);
```

PHP



fread() requires the number of bytes to read. Use filesize() to read the entire file.

</> Reading Functions

fgets()

Reads one line at a time.

Can accept optional length parameter.

```
$filehandler=fopen("welcome.txt","r");  
while (!feof($filehandler)){  
    echo fgets($filehandler). "<br>";  
}  
fclose( $filehandler);
```

fgetcsv()

- Breaks lines using a delimiter (comma, tab). Returns array. Syntax: fgetcsv(file_pointer, length, delimiter)
- Used when file is **CSV (comma-separated values)**
- **one CSV line at a time** and parses into array

```
$filehandler=fopen("welcome.txt","r");  
while (!feof($filehandler)){  
    var_dump(fgetcsv($filehandler,null, "|"));  
}  
fclose( $filehandler);
```

feof()

File End-Of-File function. Returns true if file pointer is at end of the file.



Reading File in One Step

readfile()

Returns file content as string directly to output. No fopen/fclose needed.

```
// readfile - outputs directly PHP  
readfile("welcome.txt");  
  
// return false|int the number of bytes  
read from the file, or FALSE on failure
```

file_get_contents()

Read whole file as string

@return string|false The function returns the read data or false on failure.

```
$data= file_get_contents("welcome.txt"); PHP  
var_dump($data);  
// Identical to readfile() but returns  
string instead of outputting it to  
screen direct
```



file_get_contents() is the most commonly used — simple and returns a string.

Reading from a File by `file()`

Reads file into an array.

`file(string $filename, int $flags = 0)`

- Flags:
 - `FILE_USE_INCLUDE_PATH` - Search for the file in the `include_path`.
 - `FILE_IGNORE_NEW_LINES` - Omit newline at the end of each array element
 - `FILE_SKIP_EMPTY_LINES` - Skip empty lines
- return array|false
return the file in an [array](#). Each element of the array corresponds to a line in the file, with the newline still attached.
Upon failure, file returns false.

```
// file – returns array of lines
$lines = file("welcome.txt");
var_dump($lines);
// each line is element in array
```

PHP



Processing Files (Writing)

Writing data to a file requires three steps:

1. Open the file (create if doesn't exist)
2. Write the data to the file -> Use `fwrite()` or `fputs()` (alias)
3. Close the file

```
$fh = fopen("welcome.txt", "w");  
fwrite($fh, "Hello from PHP!");  
fclose($fh);
```

PHP

`fwrite(resource $handle, string $string [, int $length])` (`$length`) is optional — max bytes to write.



Using 'w' mode will overwrite existing file contents! Use 'a' to append instead.



File Useful Functions

rewind()

Resets file pointer to beginning

fseek()

Sets file pointer to specific position

file_exists()

Checks whether a file exists

copy()

Copies file: copy(\$source, \$dest)

ftell()

Reports pointer position in bytes

f filetype()

Returns type of file (string)

unlink()

Deletes a file

basename()

Returns filename from a full path

`is_dir(), is_executable(), is_file(), is_link(), is_readable(), is_writable()`

PHP



Locking Files

Use flock() to lock a file during read/write operations.

Constant	Meaning
LOCK_SH	Reading lock — file can be shared with other readers
LOCK_EX	Writing lock — exclusive, file cannot be shared
LOCK_UN	Release the existing lock

```
if (flock($file, LOCK_EX)) {  
    fwrite($file, "Add some text");  
    flock($file, LOCK_UN); // release  
} else {  
    echo "Error locking file!";  
}  
fclose($file);
```

PHP



File Problems

Limitations of working with flat files:

Large files become very slow to process

Searching for specific records is difficult

No easy way to enforce different access levels

Concurrent access can cause bottlenecks

All operations are sequential — start to end



Solution: Use databases for structured, concurrent, and scalable data storage.



Files — Summary

`fopen()` / `fclose()`

Open and close file handles

`fread()` / `fwrite()`

Read from and write to files

`fgets()` / `fgetcsv()`

Read line by line or CSV data

`readfile()` / `file()`

Quick read — string or array

`file_get_contents()`

Read entire file as string

`flock()`

Lock files for safe concurrent access



Arrays in PHP

Indexed, Associative, and Multi-dimensional

1 2 3 Arrays in PHP

Indexed Arrays

Numerically indexed, starting at 0 by default.
Elements accessed by numeric index.

```
$arr = [3, 5, "App", true, "PHP"];
```

Associative Arrays

Key-value pairs instead of numeric indices.
Custom string keys for each element.

```
$info = ["Name" => "Mona"];
```

💡 PHP arrays are actually ordered maps — they can hold any mix of types!

```
// Creating arrays
$arr = [3, 5, "Application", true, "PHP"];
$arr1 = array("Mona", "Engineering", "ITI");
// Range function
$range = range(0, 10, 2); // [0, 2, 4, 6, 8, 10]
```

PHP

Indexed Arrays

PHP

```
$arr = [3, 5, "Application", true, "PHP"];  
// Accessing via index  
for ($i = 0; $i < count($arr); $i++) {  
    echo $arr[$i] . " ";  
}  
// Range function  
$arange = range(0, 10, 2); // Result: [0, 2, 4, 6, 8, 10]
```

PHP

```
// Looping with foreach  
$alphRange = range("A", "Z", 4);  
foreach ($alphRange as $char) {  
    echo $char . " , "; // A, E, I, M, Q, U, Y  
}
```

☰ Associative Arrays

Key-value pairs where you define custom string keys.

PHP

```
$info = [  
    "Name"    => "Mona",  
    "Email"   => "nAli@iti.gov.eg",  
    "Track"   => "Application"  
];  
// Loop through the array  
foreach ($info as $key => $value) {  
    echo $key . " : " . $value . "<br>";  
}  
// Access a specific value  
echo $info["Name"]; // Mona  
// Adding a new element  
$info["Intake"] = 35;
```



Use `compact()` to create an associative array from variables: `compact("name", "email")`

⇄ Array Operators

Operator	Name	Description
+	Union	Returns array containing everything in \$a and \$b
==	Equality	True if same key-value pairs
===	Identity	True if same key-value pairs in same order
!=	Inequality	True if \$a and \$b are not equal
<>	Inequality	Same as !=
!==	Non-identity	True if \$a and \$b are not identical

```
$num = [2, 4, 6, 8, 10];  
$alphas = ["a", "b", "c", "d"];  
$arr3 = $num + $alphas; // Union keeps first array's keys  
var_dump($arr3); // [2, 4, 6, 8, 10] - $alphas keys overlap!
```

PHP



Multi-dimensional Arrays

Each location in an array can hold another array.

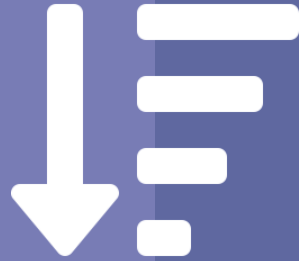
Can be indexed, associative, or mixed.

PHP

```
$students = array(  
    1 => array("Ali", "IOT"),  
    2 => array("Mostafa", "Cloud"),  
    3 => ["Mona", "Application"]  
);  
// Access: $students[1][0] → "Ali"  
// Access: $students[3][1] → "Application"
```



Multi-dimensional arrays are commonly used for database results and complex data structures.



Array Sorting

Organizing data efficiently

Array Sorting

sort()

Sorts indexed array ascending.
Case-sensitive: capitals before lowercase.
A < Z < a < z

Sort Flags

SORT_REGULAR (default)
SORT_NUMERIC
SORT_STRING

```
$names = array('Mona', "Fatma", "Dina", "Andrew", "Shimaa", "suliman");  
sort($names);  
var_dump($names);  
// Andrew, Dina, Fatma, Shimaa, Mona, suliman
```

PHP

⚠ sort() is case-sensitive! All uppercase letters sort before lowercase.

↓ Sorting Associative Arrays

asort()

Sort by VALUE, keeping key association.

```
$prices = ["meat"=>100, "sugar"=>10, "tea"=>8];  
asort($prices);
```

ksort()

Sort by KEY, keeping key association.

```
$info = ["Name"=>"Mona", ...];  
ksort($info);
```

```
// Reverse sorting functions:  
rsort() → Sort indexed array descending  
arsort() → Sort assoc. array by value descending  
krsort() → Sort assoc. array by key descending
```

PHP

💡 Use 'a' prefix to preserve key association, 'k' prefix to sort by keys.



User-defined Sort

Provide your own comparison function with `usort()`:

PHP

```
function cmp($a, $b) {  
    if ($a == $b) {  
        return 0;  
    }  
    return ($a < $b) ? -1 : 1;  
}  
$a = [3, 2, 5, 6, 1];  
// Sort using custom comparison function  
usort($a, "cmp");  
// Display the sorted array  
foreach ($a as $key => $value) {  
    echo $key . ": " . $value . "<br>";  
}  
// Output: 0:1, 1:2, 2:3, 3:5, 4:6
```

Reordering Arrays

shuffle()

Randomly reorders elements

array_reverse()

Returns copy in reverse order

array_push()

Add element to end of array

array_pop()

Remove and return last element

```
$arr = [1, 2, 3, 4, 5];  
shuffle($arr); // Random order  
$rev = array_reverse($arr); // Reversed copy  
array_push($arr, 6); // Add 6 at end  
$last = array_pop($arr); // Remove last
```

PHP

Array Flip

Exchange keys with values in associative arrays.

```
$colors = array(  
    'one'    => 'red',  
    'two'    => 'blue',  
    'three' => 'yellow'  
);
```

PHP

```
var_dump(array_flip($colors));
```

⚠ If multiple values are the same, only the last key will be preserved after flip.



Loading File into Array

Use file() function + explode() to parse CSV data into HTML tables.

PHP

```
$staff = file("csvfile.csv");

echo "<table border='2'>";
echo "<tr><th>Name</th><th>Track</th><th>Company</th></tr>";
foreach ($staff as $record) {
    $data = explode(",", trim($record)); // إزالة newline
    echo "<tr>";
    foreach ($data as $val) {
        echo "<td>" . htmlspecialchars($val) . "</td>";
    }
    echo "</tr>";
}
echo "</table>";
```



Array Navigation

Every array has an internal pointer to the current element.

in_array()

Check if value exists

current()

Get current element

next() / prev()

Move pointer fwd/back

```
$fruits = ['banana', 'apple', "Kiwi", "Orange"];
```

PHP

```
var_dump(current($fruits)); // banana
var_dump(next($fruits));   // apple
var_dump(next($fruits));   // Kiwi
var_dump(current($fruits)); // Kiwi
var_dump(reset($fruits));   // banana
var_dump(end($fruits));     // Orange
var_dump(prev($fruits));    // Kiwi
```



Array Walk

Use `array_walk()` to apply a function to every element:

PHP

```
function print_fruits($value) {  
    echo "$value <br/>";  
}  
  
$fruits = ['banana', 'apple', "Kiwi", "Orange"];  
array_walk($fruits, "print_fruits");  
// banana  
// apple  
// Kiwi
```



`array_walk()` modifies in-place. Use `array_map()` to create a new array instead.

✂ Array Merge & Chunk

array_merge()

Convert associative → indexed array.
Or merge multiple arrays together.

array_chunk()

Split array into smaller chunks.
Specify the chunk size.

```
// array_merge
$a = array(5 => "banana", 22 => "Kiwi");
var_dump(array_merge($a));
// [0 => "banana", 1 => "Kiwi"]

// array_chunk
$input = array('a', 'b', 'c', 'd', 'e');
$output = array_chunk($input, 2);
// [['a','b'], ['c','d'], ['e']]
```

PHP

Array Map & Combine

PHP

```
// array_map – loop two arrays at once
$instructors = ["Eng. Shery", "Mona", "Andrew"];
$courses = ['Admin', 'PHP', 'Node'];

$result = array_map(function($inst, $course) {
    return "$inst teaches $course";
}, $instructors, $courses);
// ["Eng. Shery teaches Admin", ...]
```

PHP

```
// array_combine – keys + values into one array
$combined = array_combine($instructors, $courses);
// ["Eng. Shery" => "Admin",
//  "Mona" => "PHP",
//  "Andrew" => "Node"]
```

💡 `array_combine()` requires both arrays to have the same number of elements.



Array Filter & Intersect

PHP

```
// array_filter – remove empty values
$arr = [1, 90, 2, null, 3, '', 55, [], 5, 6, 7, 8, ""];
$non_empty = array_filter($arr);
// [1, 90, 2, 3, 55, 5, 6, 7, 8]
```

PHP

```
// array_intersect_key – compare by keys
$a1 = ['blue'=>1, 'red'=>2, 'green'=>3, 'purple'=>4];
$a2 = ['green'=>5, 'blue'=>6, 'yellow'=>7, 'cyan'=>8];
var_dump(array_intersect_key($a1, $a2));
// ['blue'=>1, 'green'=>3]
// Filter specific keys
$arr = ['a'=>123, 'b'=>213, 'c'=>321];
$allowed = ['b', 'c'];
array_intersect_key($arr, array_flip($allowed));
// ['b'=>213, 'c'=>321]
```



Count Elements

PHP

```
$students = ["Ali", "Ahmed", "Mostafa", "Omar", "Ahmed"];

var_dump(count($students)); // 5
var_dump(sizeof($students)); // 5 (alias)

// Count occurrences of each value
var_dump(array_count_values($students));
// ['Ali'=>1, 'Ahmed'=>2, 'Mostafa'=>1, 'Omar'=>1]
```

💡 sizeof() is an alias of count(). Both work identically.

⇄ Convert Array to Scalars

Convert array elements into individual variables:

extract()

Associative array → variables

```
// extract() – associative array PHP  
$info = ["username"=>"Mona",  
        "email"=>"nAli@iti.gov.eg",  
        "track"=>"Application"];  
extract($info);  
  
echo $username . " " . $email . " " .
```

list()

Indexed array → variables

```
// list() – indexed array PHP  
$info = ['coffee', 'brown',  
        'caffeine'];  
list($drink, $color, $power) =  
$info;
```

⚠ extract() can overwrite existing variables! Use EXTR_PREFIX_ALL flag for safety.



Arrays — Summary

Indexed	Numeric keys, range(), for/foreach loops
Associative	Custom string keys, key=>value pairs
Multi-dimensional	Arrays within arrays, nested access
Sorting	sort, asort, ksort, rsort, usort
Navigation	current, next, prev, reset, end
Functions	array_merge, chunk, map, filter, combine



Lab 02

Registration Form Exercise



Lab 02 — Registration Form

- Add the server-side validation to the form fields. (firstname,lastname,email,gender) • When you submit the data, Save it the customer.txt file.
- Retrieve all the records stored in the file and display them in a table.
- Add the delete button in the table, when you click it, record deleted from the table and file also

Search about array function and tell me functionality about 5 function



Thank You!

Day 02 Complete — Files & Arrays

Next: Day 03 — Functions & String Manipulation

Prepared by: Mona Ali — ITI